

01_TOKENS

Main.java

```
1 import java.util.LinkedHashMap;
2 import java.util.Map;
3
4 public class Main {
5     static final String ALUNO = "LEONARDO FRATONI BORGES DA SILVA";
6     static final String RA = "24047610-2";
7
8     static final String ENTRADA = "total_lf = valor + 02;";
9
10    private static final Map<Character, String> SIMBOLOS = new LinkedHashMap<>();
11
12    static {
```

Console

I/O

AI Agent

939 ms

ALUNO: LEONARDO FRATONI BORGES DA SILVA | RA: 24047610-2

ENTRADA: total_lf = valor + 02;

LEXEMA	TOKEN
total_lf	IDENTIFICADOR
=	ATRIBUICAO
valor	IDENTIFICADOR
+	SOMA
02	INTEIRO
;	PONTO_E_VIRGULA

02_LEXICO

Main.java

```
1 import java.util.Arrays;
2 import java.util.HashSet;
3 import java.util.Set;
4
5 public class Main {
6     static final String ALUNO = "LEONARDO_FRATONI_BORGES_DA_SILVA";
7     static final String RA = "24047610-2";
8
9     static final String IDENTIFICADOR_PERSONALIZADO = "nota_lf";
10
11     static final Set<String> PALAVRAS_RESERVADAS = new HashSet<>()
12     Arrays.asList("int", "if", "else", "while", "return");
13
14
15     public static void main(String[] args) {
16         System.out.println("ALUNO: " + ALUNO + " | RA: " + RA);
17         executarTeste("TESTE A - entrada compacta",
18             "int total=valor+10;");
19         executarTeste("TESTE B - espacos e comentario",
20             "int total = valor + 10; // este comentario sera ignorado");
21         executarTeste("TESTE C - reservadas e identificadores",
22             "int inteiro = 0; while (inteiro < intValor) inteiro = inteiro + 1;");
23         executarTeste("TESTE C2 - identificador personalizado",
24             "int " + IDENTIFICADOR_PERSONALIZADO + " = 10;");
25         executarTeste("TESTE D - caractere inválido",
26             "int valor# = 1;");
27     }
28 }
```

Console

I/O

AI Agent

1.6s

inteiro

LETRA

IDENTIFICADOR

+

SIMBOLO

SOMA

1

DIGITO

INTEIRO

;

SIMBOLO

PONTO_E_VIRGULA

=== TESTE C2 - identificador personalizado ===

ENTRADA: int nota_lf = 10;

LEXEMA

CLASSE

TOKEN

int

LETRA

PALAVRA_RESERVADA

nota_lf

LETRA

IDENTIFICADOR

=

SIMBOLO

ATRIBUICAO

10

DIGITO

INTEIRO

;

SIMBOLO

PONTO_E_VIRGULA

=== TESTE D - caractere inválido ===

ENTRADA: int valor# = 1;

LEXEMA

CLASSE

TOKEN

int

LETRA

PALAVRA_RESERVADA

valor

LETRA

IDENTIFICADOR

#

DESCONHECIDO

ERRO_LEXICO

=

SIMBOLO

ATRIBUICAO

1

DIGITO

INTEIRO

;

SIMBOLO

PONTO_E_VIRGULA

03_ERRO_COMPILADOR

Main.java

```
1 public class Main {
2     static final String ALUNO = "SUBSTITUA_PELo_SEU_NOME";
3     static final String RA = "00000000";
4
5     public static void main(String[] args) {
6         System.out.println("ALUNO: " + ALUNO + " | RA: " + RA);
7
8         int total = 10 + 20 // CORRIGIR 1: acrescente o ponto e virgula
9
10        if (total > 20 { // CORRIGIR 2: feche corretamente o parenteses
11            System.out.println("Programa corrigido!"); // CORRIGIR 3: feche o texto entre aspas
12            System.out.println("Resultado: " + total);
13        }
14    }
15 }
```

Main.java:8: error: ';' expected

int total = 10 + 20 // CORRIGIR 1: acrescente o ponto e virgula

^

Main.java:10: error: ')' expected

if (total > 20 { // CORRIGIR 2: feche corretamente o parenteses

^

Main.java:11: error: unclosed string literal

System.out.println("Programa corrigido!"); // CORRIGIR 3: feche o texto entre aspas

^

3 errors

04_PROGRAMA_CORRIGIDO

```
1 public class Main {
2     static final String ALUNO = "LEONARDO_FRATONI_BORGES_DA_SILVA";
3     static final String RA = "24047610-2";
4
5     public static void main(String[] args) {
6         System.out.println("ALUNO: " + ALUNO + " | RA: " + RA);
7
8         int total = 10 + 20; // CORRIGIR 1: acrescente o ponto e virgula
9
10        if (total > 20) { // CORRIGIR 2: feche corretamente o parenteses
11            System.out.println("Programa corrigido!"); // CORRIGIR 3: feche o texto entre
12            System.out.println("Resultado: " + total);
13        }
14    }
15 }
```

ALUNO: LEONARDO_FRATONI_BORGES_DA_SILVA | RA: 24047610-2
Programa corrigido!
Resultado: 30

05_DESCENTE

```
1 public class Main {
2     static final String ALUNO = "LEONARDO_FRATONI_BORGES_DA_SILVA";
3     static final String RA = "24047610-2";
4
5     static final String ENTRADA = "(2 + 3) * 4";
6     static final boolean DEMONSTRAR_RECURSAO_ESQUERDA = false;
7
8     public static void main(String[] args) {
9         System.out.println("ALUNO: " + ALUNO + " | RA: " + RA);
10        System.out.println("ENTRADA: " + ENTRADA);
11        System.out.println();
12
13        if (DEMONSTRAR_RECURSAO_ESQUERDA) {
14            demonstrarRecursoaEsquerda(0);
15            System.out.println();
16        }
17
18        try {
19            Parser parser = new Parser(ENTRADA);
20            int resultado = parser.analisar();
21            System.out.println();
22            System.out.println("ENTRADA ACEITA");
23            System.out.println("RESULTADO DA EXPRESSAO: " + resultado);
24        } catch (ErroSintatico erro) {
25            System.out.println();
26            System.out.println("ERRO SINTATICO: " + erro.getMessage());
27        }
28    }
29
30    static void demonstrarRecursoaEsquerda(int nivel) {
31        System.out.println(recuo(nivel)
32        | | | + "expr -> expr + term (chamada " + (nivel + 1) + ")");
33
34        if (nivel == 7) {
```

Novo lookahead: 2
ENTRANDO em <expr> | lookahead = 2
ENTRANDO em <term> | lookahead = 2
ENTRANDO em <factor> | lookahead = 2
Numero reconhecido: 2
Novo lookahead: +
SAINDO de <factor> | lookahead = +
SAINDO de <term> | lookahead = +
Encontrado operador +
Novo lookahead: 3
ENTRANDO em <term> | lookahead = 3
ENTRANDO em <factor> | lookahead = 3
Numero reconhecido: 3
Novo lookahead:)
SAINDO de <factor> | lookahead =)
SAINDO de <term> | lookahead =)
SAINDO de <expr> | lookahead =)
Novo lookahead: *
Fechando parenteses
SAINDO de <factor> | lookahead = *
Encontrado operador *
Novo lookahead: 4
ENTRANDO em <factor> | lookahead = 4
Numero reconhecido: 4
Novo lookahead: \$FIM\$
SAINDO de <factor> | lookahead = \$FIM\$
SAINDO de <term> | lookahead = \$FIM\$
SAINDO de <expr> | lookahead = \$FIM\$

ENTRADA ACEITA
RESULTADO DA EXPRESSAO: 20

06_ASCENDENTE

ALUNO: LEONARDO_FRATONI_BORGES_DA_SILVA | RA: 24047610-2
ENTRADA: id+id*id

PILHA	ENTRADA	ACAO

\$ 0	id+id*id\$	SHIFT id ->
estado 5		
\$ 0 id 5	+id*id\$	REDUCE F -> id
\$ 0 F 3	+id*id\$	REDUCE T -> F
\$ 0 T 2	+id*id\$	REDUCE E -> T
\$ 0 E 1	+id*id\$	SHIFT + ->
estado 6		
\$ 0 E 1 + 6	id*id\$	SHIFT id ->
estado 5		
\$ 0 E 1 + 6 id 5	*id\$	REDUCE F -> id
\$ 0 E 1 + 6 F 3	*id\$	REDUCE T -> F
\$ 0 E 1 + 6 T 9	*id\$	SHIFT * ->
estado 7		
\$ 0 E 1 + 6 T 9 * 7	id\$	SHIFT id ->
estado 5		
\$ 0 E 1 + 6 T 9 * 7 id 5	\$	REDUCE F -> id
\$ 0 E 1 + 6 T 9 * 7 F 10	\$	REDUCE T -> T *
F		
\$ 0 E 1 + 6 T 9	\$	REDUCE E -> E +
T		
\$ 0 E 1	\$	ACCEPT